HUNT-SAGE · THREAT HUNTING CHEAT SHEET

Linux Logs & auditd for Hunters

Where evidence lives on modern Linux (journald, auditd, eBPF, containers), a copy-ready auditd ruleset, triage one-liners, SSH log patterns and the execution patterns worth stacking.

github.com/iamjammie/hunt-sage · iamjammie.github.io/hunt-sage · Built 2026-11-03

Linux telemetry is usually the weakest in an estate. Hunt #0 is *confirm you have `execve` auditing or an EDR on your Linux servers*; the Linux hunting playbook starts there. Paths below are for RHEL 9/10, Ubuntu 22.04/24.04, Debian 12 and Amazon Linux 2023; Debian 12 and Amazon Linux 2023 ship **journald only** (no `/var/log/syslog` or `/var/log/messages` unless rsyslog is installed).

Where to look

Evidence	Where	Notes
Everything, structured	`journalctl`	`-u sshd` (RHEL) or `-u ssh` (Ubuntu), `_COMM=sudo`, `_UID=1000`, `-p warning`, `-k` (kernel), `--since "2 hours ago"`, `--list-boots`, `-o json`, `--grep 'Failed'`. Persist it: `Storage=persistent` in `/etc/systemd/journald.conf` or journals vanish on reboot
Authentication, sudo, SSH, PAM	`/var/log/auth.log` (Debian/Ubuntu), `/var/log/secure` (RHEL)	`Accepted` / `Failed password`, `publickey`, `sudo:` lines, `pam_unix(sshd:session)`, `useradd`, `usermod`, `passwd`
General syslog	`/var/log/syslog` (Ubuntu), `/var/log/messages` (RHEL)	Cron on Ubuntu lands here (`CRON[...]`)
Logins	`last`, `lastb` (`/var/log/wtmp`, `/var/log/btmp`), `lastlog`, `who` / `w` (`/run/utmp`)	Binary files: `last -f`. Newer distributions are moving to `wtmpdb` / `lastlog2`; absence of a session in wtmp that auditd saw is itself a lead
sudo	`auth.log` / `secure`, journald `_COMM=sudo`	`Defaults log_output` records full sessions to `/var/log/sudo-io` (replay with `sudoreplay`); sudo 1.9 adds `log_format=json`, `log_subcmds`, and central `sudo_logsrvd`
Cron	`/var/log/cron` (RHEL), `syslog` (Ubuntu), `journalctl -u crond` / `-u cron`	Files: `/etc/crontab`, `/etc/cron.*`, `/var/spool/cron/` (RHEL) or `/var/spool/cron/crontabs/` (Debian), `/etc/anacrontab`
systemd units and timers	`systemctl list-timers --all`, `systemctl list-unit-files --state=enabled`, `journalctl -u <unit>`	Unit dirs: `/etc/systemd/system/`, `/run/systemd/system/` (transient, from `systemd-run`), `/usr/lib/systemd/system/`, user units in `~/.config/systemd/user/`; generators in `/etc/systemd/system-generators/`
Preload hijacks	`/etc/ld.so.preload`, `/etc/ld.so.conf.d/`, `LD_PRELOAD` in `/proc/<pid>/environ`	A non-empty `ld.so.preload` is almost always worth a look (T1574.006)
Shell history	`~/.bash_history`, `~/.zsh_history`, `~/.python_history`, `~/.mysql_history`, `~/.viminfo`, `~/.lesshst`	Written on shell exit; defeated by `HISTFILE=/dev/null`, `HISTSIZE=0`, `set +o history`, a leading space with `HISTCONTROL=ignorespace`. Absence or truncation is a signal; auditd `execve` is the reliable record
Package changes	`/var/log/apt/history.log`, `/var/log/apt/term.log`, `/var/log/dpkg.log`; `/var/log/dnf.log`, `/var/log/dnf.rpm.log`, `dnf history`; `rpm -qa --last`	Verify binaries against the package database: `rpm -Va`, `dpkg -V` or `debsums -c`; `pip` / `npm` installs leave nothing here
Kernel	`dmesg -T`, `journalctl -k`, `/var/log/kern.log` (Ubuntu)	Module loads, `taint` changes (`/proc/sys/kernel/tainted`), USB device insertion, OOM kills, segfaults of system daemons

Evidence	Where	Notes
Live process state	`/proc/<pid>/{exe,cmdline,environ,cwd,fd,maps,status}`	`exe -> ... (deleted)` means the binary was removed after start; `TracerPid` in `status` reveals a debugger; `/proc/<pid>/comm` can be faked, `exe` cannot
Containers	`docker logs`, `/var/lib/docker/containers/<id>/<id>-json.log`, `journalctl -u containerd`, `/var/log/pods/`, `/var/log/containers/`, kubelet journal	Host-side auditd sees container processes too (with cgroup context missing); see the Kubernetes playbook for API audit logs
Cloud bootstrap	`/var/log/cloud-init.log`, `/var/log/cloud-init-output.log`	User-data scripts are a persistence and initial-access vector on cloud VMs
Web servers	`/var/log/apache2/`, `/var/log/httpd/`, `/var/log/nginx/`	Web shells and exploitation; pair with the web shells playbook
auditd	`/var/log/audit/audit.log`	The kernel audit subsystem; `ausearch`, `aureport`, `auditctl`. Covered below
eBPF sensors	Tetragon, Falco, Tracee (and most EDRs)	Add what auditd lacks: full process lineage, container and pod identity, file and network events with low overhead, no backlog loss, TTY capture, and in Tetragon's case enforcement
osquery	`processes`, `process_events`, `socket_events`, `listening_ports`, `crontab`, `systemd_units`, `authorized_keys`, `users`, `groups`, `suid_bin`, `kernel_modules`, `shell_history`, `rpm_packages` / `deb_packages`, `docker_containers`	Scheduled packs give fleet-wide stacking; `*_events` tables need the audit or eBPF publisher
Sysmon for Linux	syslog, XML payload	A subset of Windows Sysmon: 1 ProcessCreate, 3 NetworkConnect, 5, 9, 10, 11 FileCreate, 16, 23 FileDelete; needs the `SysinternalsEBPF` library

auditd essentials

Thing	What to know
Record types	`SYSCALL` (the call, uid/auid/exe/key), `EXECVE` (argv, one `a0..aN` per argument), `PATH` (files touched), `CWD`, `PROCTITLE` (full command line, hex unless `-i`), `USER_AUTH` / `USER_LOGIN` / `USER_START` / `USER_END` (PAM), `USER_CMD` (sudo), `CRED_ACQ` / `CRED_DISP`, `CONFIG_CHANGE`, `SERVICE_START` / `STOP`, `SOCKADDR`, `KERN_MODULE`, `ANOM_ABEND`, `ANOM_PROMISCUOUS`, `AVC` (SELinux denials), `SYSTEM_BOOT` / `SHUTDOWN`, `DAEMON_START`
`auid` vs `uid`	`auid` (login uid) is set at login and survives `su` / `sudo`; `uid` / `euid` are the effective identities. `auid=4294967295` (`unset`, `-1`) means a daemon or something that never logged in. `ses` ties records to a login session
Key fields	`exe` (binary path), `comm` (process name, 15 chars), `key` (your rule tag), `success=yes/no`, `exit` (return code), `syscall` (number; `ausyscall --dump` or `ausyscall x86_64 59` to translate), `a0..a3` (raw arguments), `ppid`, `tty`, `subj` (SELinux context)
Searching	`ausearch -i` interprets uids and hex; `-k <key>`, `-ts today` / `recent` / `boot` / `this-week`, `-m EXECVE`, `-ua 1000` (auid), `-x /usr/bin/curl`, `-c bash`, `-sc execve`, `-p <pid>`, `-sv no`, `--format csv`, `-if <file>` for an offline log
Reporting	`aureport -x --summary` (executables), `-au` (auth attempts), `-l` (logins), `-u` (users), `-f` (files), `-k` (keys), `-n` (anomalies), `-ts today`; `aureport -x -i --summary` is a fast fleet stack
Rules files	`/etc/audit/rules.d/*.rules`, merged by `augenrules --load` into `/etc/audit/audit.rules`; `auditctl -l` shows what is active, `auditctl -s` shows status, backlog and lost counts
Immutability	`-e 2` as the last line makes rules immutable until reboot; `--loginuid-immutable` stops `auid` tampering. Changing rules then needs a reboot, which is the point
Volume and loss	`-b 8192` (backlog), `--backlog_wait_time`, `-f 1` (print on failure), `-r 0` (no rate limit); watch `auditctl -s` for `lost`. `auditd.conf`: `log_format = ENRICHED`, `max_log_file`, `num_logs`, `max_log_file_action = ROTATE`, `space_left_action`, `disk_full_action`, `flush = incremental_async`
Forwarding	Ship `audit.log` with an agent (Filebeat `auditd` module, Splunk TA for Linux, Azure Monitor Agent, Elastic Auditbeat replacing auditd), or the `au-remote` / `syslog` plugins in `/etc/audit/plugins.d/`. Prefer structured shipping; raw multi-line records are painful to parse
Containers	Audit is kernel-wide and not namespaced: one auditd on the host sees every container's syscalls, containers cannot run their own, and records carry no pod identity. Use an eBPF sensor for container context

Starter auditd rules (`/etc/audit/rules.d/hunt.rules`)

Derived from the Neo23×0 (Florian Roth) `auditd` ruleset and the CIS and DISA STIG baselines; trim to your environment, then test volume for a week before enforcing immutability.

HUNT-SAGE
iamjammie/hunt-sage

```
-D
-b 8192
-f 1
--backlog_wait_time 60000
-i

# Process execution (the core; 64-bit and 32-bit ABI)
-a always,exit -F arch=b64 -S execve -k exec
-a always,exit -F arch=b32 -S execve -k exec
# Execution from writable staging areas
-a always,exit -F arch=b64 -S execve -F dir=/tmp -k exec_tmp
-a always,exit -F arch=b64 -S execve -F dir=/var/tmp -k exec_tmp
-a always,exit -F arch=b64 -S execve -F dir=/dev/shm -k exec_tmp

# Identity and privilege
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/group -p wa -k identity
-w /etc/gshadow -p wa -k identity
-w /etc/sudoers -p wa -k sudoers
-w /etc/sudoers.d/ -p wa -k sudoers
-w /usr/bin/sudo -p x -k priv_cmd
-w /usr/bin/su -p x -k priv_cmd
-w /usr/bin/passwd -p x -k priv_cmd
-a always,exit -F arch=b64 -S setuid,setgid,setreuid,setresuid -F auid>=1000 -F auid!=unset -k priv_esc

# Persistence: cron, systemd, SSH, preload
-w /etc/crontab -p wa -k cron
-w /etc/cron.d/ -p wa -k cron
-w /etc/cron.daily/ -p wa -k cron
-w /var/spool/cron/ -p wa -k cron
-w /etc/systemd/system/ -p wa -k systemd
-w /usr/lib/systemd/system/ -p wa -k systemd
-w /etc/ssh/sshd_config -p wa -k sshd
-w /etc/ssh/sshd_config.d/ -p wa -k sshd
-w /root/.ssh/ -p wa -k ssh_keys
-w /etc/ld.so.preload -p wa -k ld_preload
-w /etc/ld.so.conf.d/ -p wa -k ld_preload
-w /etc/profile.d/ -p wa -k shell_profile
-w /etc/bash.bashrc -p wa -k shell_profile

# Kernel modules, tracing, mounts
-a always,exit -F arch=b64 -S init_module,finit_module,delete_module -k modules
-w /usr/sbin/insmod -p x -k modules
-w /usr/sbin/modprobe -p x -k modules
-a always,exit -F arch=b64 -S ptrace -F a0=0x4 -k code_injection
-a always,exit -F arch=b64 -S ptrace -F a0=0x5 -k data_injection
-a always,exit -F arch=b64 -S ptrace -F a0=0x10 -k ptrace_attach
-a always,exit -F arch=b64 -S mount,umount2 -F auid>=1000 -F auid!=unset -k mounts

# Permission changes on anything by real users (volume: medium)
-a always,exit -F arch=b64 -S chmod,fchmod,fchmodat,chown,fchown,fchownat,lchown -F auid>=1000 -F auid!=unset -k per
m_mod

# Network configuration and the container socket
-w /etc/hosts -p wa -k network
-w /etc/resolv.conf -p wa -k network
-w /etc/netplan/ -p wa -k network
-w /etc/NetworkManager/ -p wa -k network
-a always,exit -F arch=b64 -S sethostname,setdomainname -k network
-w /run/docker.sock -p rwxa -k docker
-w /usr/bin/docker -p x -k docker
-w /usr/bin/kubectl -p x -k kube

# Package managers (new tooling arriving)
-w /usr/bin/dpkg -p x -k package
-w /usr/bin/apt -p x -k package
-w /usr/bin/apt-get -p x -k package
-w /usr/bin/dnf -p x -k package
-w /usr/bin/rpm -p x -k package
-w /usr/bin/pip3 -p x -k package

# Time and audit integrity
-a always,exit -F arch=b64 -S adjtimex,settimeofday,clock_settime -k time_change
-w /etc/localtime -p wa -k time_change
-w /etc/audit/ -p wa -k audit_config
-w /sbin/auditctl -p x -k audit_tools
-w /sbin/auditd -p x -k audit_tools

# Noise control: drop the chatty, low-value records
-a exclude,always -F msgtype=CWD
-a exclude,always -F msgtype=CRYPTO_KEY_USER
```

```
-a never,exit -F arch=b64 -S adjtimex -F uid=chrony -k time_change

# Make the configuration immutable until reboot (keep last)
-e 2
```

Load with `augenrules --load`, verify with `auditctl -l`. `-w` does not take wildcards, so per-user `~/.ssh` monitoring needs one rule per home or a file-system watcher. `-S connect` auditing is possible but expensive; use an eBPF sensor or Sysmon for Linux for network events.

Triage one-liners

```
# auditd: what ran, by whom, from where
ausearch -k exec -ts today -i | grep -E 'proctitle|auid' | head -50
aureport -x -i --summary -ts this-week | head -40
ausearch -k exec_tmp -ts recent -i
ausearch -m USER_LOGIN -sv no -ts today -i          # failed logins seen by PAM
ausearch -k priv_esc -i | grep -E 'exe=|auid='
# Logins and sessions
w; last -aiF | head -30; lastb -aiF | head -30
journalctl -u sshd --since "24 hours ago" --grep 'Accepted|Failed|Invalid' --no-pager
journalctl _COMM=sudo --since today --no-pager
# Failed SSH by source and by username
journalctl -u sshd --since "7 days ago" -o cat | grep 'Failed password' | awk '{print $(NF-3)}' | sort | uniq -c | s
ort -rn | head
journalctl -u sshd --since "7 days ago" -o cat | grep 'Invalid user' | awk '{print $(NF-3)}' | sort | uniq -c | sort
-rn | head
# Accounts and groups
awk -F: '$3==0' /etc/passwd                          # every UID 0
awk -F: '$7!~/nologin|false/' /etc/passwd            # accounts that can log in
getent group sudo wheel docker adm                   # who can escalate
grep -E 'useradd|usermod|groupadd' /var/log/auth.log /var/log/secure 2>/dev/null | tail
# SUID/SGID, world-writable, recently changed binaries in PATH
find / -xdev -type f \( -perm -4000 -o -perm -2000 \) -ls 2>/dev/null
find / -xdev -type d -perm -0002 -not -path '/proc/*' -not -path '/tmp*' -ls 2>/dev/null | head
find /usr/bin /usr/sbin /usr/local/bin -type f -newermt "$(date -d '-7 days' +%F)" -ls 2>/dev/null
rpm -Va 2>/dev/null | grep -E '^..5' | head          # RHEL: modified package files
debsums -c 2>/dev/null | head                        # Debian/Ubuntu (apt install debsums)
# Persistence inventory
systemctl list-timers --all --no-pager; systemctl list-unit-files --state=enabled --no-pager | head -60
for u in $(cut -d: -f1 /etc/passwd); do crontab -l -u "$u" 2>/dev/null | sed "s/^/$u: /"; done
ls -la /etc/cron.* /var/spool/cron* /etc/systemd/system /etc/profile.d 2>/dev/null
find / -xdev -name authorized_keys -ls 2>/dev/null; cat /etc/ld.so.preload 2>/dev/null
# Processes: deleted binaries, writable staging dirs, preload, orphans
ls -l /proc/[0-9]*/exe 2>/dev/null | grep -E 'deleted|/tmp/|/dev/shm/|/var/tmp/'
grep -l LD_PRELOAD /proc/[0-9]*/environ 2>/dev/null
ps -eo pid,ppid,user,etimes,comm,args --forest | less
for p in /proc/[0-9]*; do e=$(readlink $p/exe 2>/dev/null) && [ -n "$e" ] && ! rpm -qf "$e" >/dev/null 2>&1 && echo
"$p $e"; done   # RHEL: binaries no package owns
# Network: listeners and established connections with the owning process
ss -tulpn; ss -tanp state established | grep -vE '127\.0\.0\.1|::1'
# Containers from the host side
docker ps 2>/dev/null
docker inspect $(docker ps -q) 2>/dev/null | jq -r '.[] | [.Name, "privileged=\(.HostConfig.Privileged)", "pid=\(.Ho
stConfig.PidMode)", ([.Mounts[]?.Source] | join(","))] | @tsv'
```

SSH log patterns

Set `LogLevel VERBOSE` in `sshd_config`: it adds the key fingerprint on every accept and logs port-forwarding requests, which the default `INFO` level does not.

Line (abridged)	Meaning / hunt use
`Accepted publickey for alice from 203.0.113.9 port 51234 ssh2: ED25519 SHA256:...`	Key-based login; the fingerprint is the identity. Map fingerprints to owners; an unknown fingerprint on a known account is a planted key (T1098.004)
`Accepted password for root from ...`	Password root login; should be impossible (`PermitRootLogin prohibit-password`)
`Accepted certificate ID "..." (serial N) signed by ... CA`	SSH certificate auth (`TrustedUserCAKeys`); the CA and serial are the pivot
`Failed password for alice from ...` / `Failed password for invalid user admin from ...`	Guessing; `invalid user` means the account does not exist, a spray signature
`Invalid user oracle from 198.51.100.7 port ...`	Username enumeration; stack by source

Line (abridged)	Meaning / hunt use
`error: maximum authentication attempts exceeded for ...`	`MaxAuthTries` hit; brute force
`Disconnected from authenticating user root 198.51.100.7 port ... [preauth]` / `Connection closed by ... [preauth]`	Scanner or spray that gave up before auth
`Timeout before authentication for ...`	Banner grabbing, slow scanners
`pam_unix(sshd:session): session opened for user alice(uid=1000) by (uid=0)`	Session start; pair with `session closed` for duration
`Received disconnect from ...: 11: disconnected by user`	Normal client logout
`Received request to connect to host 10.0.0.5 port 3389, but the request was denied`	Blocked `-L` forward (`AllowTcpForwarding no`); an allowed one is logged only at VERBOSE as `Connection to ... forwarded`
`remote forwarding listen ...` / `Forwarding listen address` (VERBOSE)	`ssh -R` reverse tunnel: an inbound pivot through your server (T1572)
`reverse mapping checking getaddrinfo for ... failed`	Source has no matching PTR; common on hostile sources, not conclusive
`subsystem request for sftp`	SFTP session; large transfers here are exfil candidates
`Server listening on 0.0.0.0 port 22` with an unexpected port or time	sshd restarted; config change

Pair with `ssh.log` from Zeek (direction, client string, `auth_success`) in the network sheet and the lateral-movement hypotheses in the playbook.

Suspicious execution patterns (leads, not verdicts)

Pattern	Why it matters	ATT&CK	Shows up in
`curl ...	sh` , `wget -qO- ...	bash` , `curl -fsSL ...	sudo bash`	Pipe-to-shell staging from the internet	T1059.004, T1105	auditd `-k exec` (`EXECVE` a0..aN), eBPF exec events
`echo ...	base64 -d	sh` , `python3 -c "import base64..."` , `xxd -r -p`	Obfuscated execution	T1027, T1140	`-k exec` ; stack `comm` + argument length
Execution from `/tmp` , `/var/tmp` , `/dev/shm` , `/run/user/*` , a web root	Writable staging areas	T1036	`-k exec_tmp` ; `/proc/<pid>/exe`
`chmod +x` then execution of the same path within seconds	Dropper	T1222.002	`-k perm_mod` followed by `-k exec` on the same `PATH` name
`nohup` , `setsid` , `disown` , `screen -dmS` , `tmux new -d` wrapping network tools	Backgrounded implant	T1059.004	`-k exec` ; `ps -o etimes` for long-lived oddities
`python3 -c 'import pty; pty.spawn("/bin/bash")'` , `script /dev/null` , `socat ... exec:bash,pty`	TTY upgrade after a reverse shell	T1059.004	`-k exec` : an interpreter (`python3` , `perl` , `php` , `java`) as the parent of `bash` / `sh`
`bash -i >& /dev/tcp/...` , `nc -e` , `mkfifo` + `nc` , `socat tcp:...`	Reverse shell primitives	T1059.004	`-k exec` ; shells whose parent is a web server or a service account (`auid=unset` , `uid=www-data`)
`find / -perm -4000` , `getcap -r /` , `sudo -l` , `id; uname -a; cat /etc/os-release` in quick succession	Enumeration (linpeas, LinEnum, pspy leave these fingerprints)	T1082, T1083	`-k exec` ; many discovery commands from one `ses` in under a minute
`systemctl stop auditd` / `rsyslog` , `auditctl -e 0` , `service auditd stop` , `iptables -F` , `setenforce 0`	Defence evasion	T1562.001, T1562.012	`-k audit_tools` , `-k audit_config` , `CONFIG_CHANGE` , `SERVICE_STOP` records
`history -c` , `export HISTFILE=/dev/null` , `unset HISTFILE` , `shred` / `truncate` on logs, `rm /var/log/*`	Anti-forensics	T1070.003, T1070.002	`-k exec` (builtins like `history -c` only show in `PROCTITLE` of the shell); `PATH` records for `/var/log`
`useradd` with `-o -u 0` , `usermod -aG sudo` , edits to `/etc/sudoers.d/`	Account persistence	T1136.001, T1098	`-k identity` , `-k sudoers` , `ADD_USER` / `ADD_GROUP` records
`crontab -` , `tee /etc/cron.d/...` , `systemctl enable` on a unit in `/etc/systemd/system/` created minutes earlier, `at now + 1 minute`	Scheduled persistence	T1053.003, T1543.002	`-k cron` , `-k systemd` ; outside a change window

Pattern	Why it matters	ATT&CK	Shows up in
New SUID/SGID file, `chmod u+s`, `setcap cap_setuid+ep`	Privilege persistence	T1548.001	`-k perm_mod` with `a1` mode bits including `04000`; `find -perm -4000` diff against last week
`insmod`, `modprobe` of a module outside `/lib/modules`, `kernel: ... taints kernel`	Rootkit or driver load	T1014, T1547.006	`-k modules`, `KERN_MODULE`, `journalctl -k` taint messages
`nsenter -t 1 -m -u -i -n`, `docker run --privileged -v /:/host`, `mount /dev/sda1`, writes to `/var/run/docker.sock`	Container escape	T1611	`-k docker`, `-k mounts`; host auditd sees the container's `execve`
`xmrig`, `kinsing`, `kdevtmpfsi`, `/tmp/.X11-unix/...`, high CPU with a deleted exe	Cryptomining	T1496	`-k exec_tmp`; `/proc/<pid>/exe (deleted)`; CPU metrics
`ssh -R`, `ssh -D`, `chisel`, `ligolo`, `frpc`, `ngrok`, `socat ... fork`	Tunnelling and pivoting	T1572, T1090	`-k exec`; sshd VERBOSE forwarding lines; `ss -tanp` listeners owned by user processes

Pull Linux into your hunt platform

Route	Notes
auditd to Splunk	Splunk Add-on for Unix and Linux: `sourcetype=linux_audit` (one event per record; join on `msg=audit(ts:serial)`), `linux_secure` for auth. Field names follow auditd (`type`, `key`, `auid`, `exe`, `comm`, `proctitle`)
auditd to Sentinel	Azure Monitor Agent Syslog DCR for `auth` / `authpriv` / `cron` facilities into `Syslog`; the Linux auditd solution or a custom DCR for `audit.log`; Defender for Endpoint on Linux gives `DeviceProcessEvents` with full lineage
journald	Forward with `systemd-journal-remote` / `-upload`, Vector, Fluent Bit (`systemd` input) or `journalctl -o json` into any collector; fields are `_COMM`, `_EXE`, `_UID`, `_PID`, `MESSAGE`, `SYSLOG_IDENTIFIER`
osquery	Scheduled packs (`processes`, `listening_ports`, `crontab`, `systemd_units`, `authorized_keys`, `suid_bin`) via Fleet or Kolide; results land as JSON snapshots, ideal for fleet-wide stacking
Falco / Tetragon	Falco rules emit JSON (`rule`, `priority`, `output_fields.proc.cmdline`, `container.id`, `k8s.pod.name`); Tetragon exports `process_exec` / `process_kprobe` events with full lineage and pod labels via `tetra getevents -o json` or the export file
Field mapping	Keep one vocabulary: `auid` = user.audit.id = initiating user; `exe` = process.executable; `PROCTITLE` = process.command_line; `ppid` = process.parent.pid; `ses` = session id; container id from cgroup in eBPF sources

```
sourcetype=linux_audit type=EXECVE earliest=-30d
| rex field=_raw "a0=\"?(?<a0>[^\" ]+)"
| stats dc(host) as hosts count earliest(_time) as first_seen by a0
| where hosts<=2 AND first_seen>relative_time(now(),"-1d")
| sort -first_seen
```

```
Syslog
| where TimeGenerated > ago(7d) and Facility in ("auth", "authpriv") and ProcessName == "sshd"
| where SyslogMessage startswith "Accepted"
| parse SyslogMessage with "Accepted " Method " for " User " from " SourceIp " port " Port " ssh2: " KeyType " " Fin
gerprint
| summarize Hosts = dcount(Computer), FirstSeen = min(TimeGenerated), Users = make_set(User, 10) by Fingerprint, Key
Type
| where FirstSeen > ago(1d)
```

More in the query index and the Linux hunting playbook; the reverse-shell Sigma rule is in research.